# A kernel-writing agent under an 8k-token budget

Demo notes, Team 27 (seat 130) • Project 02, Stage A

2026-10-10 • PR #20: `projects/team-27-kernel-agent/` in the fork
`https://github.com/LiYou22/trainium-agent-labs` (branch `team-27-kernel-agent`)

## 1. The problem and our claim

Qwen3-8B (vLLM on a Trainium seat pod, 8192 tokens per call including the answer) must write tiled NumPy kernels for a 10-level ladder (relu … conv1d) under chip-like rules: tiles of at most $128 \times 512$, explicit loops, no fancy indexing or broadcasting, no calling the op itself, partial tiles handled. The dilemma: rules in the prompt make the small model audit itself into silence; no rules and it writes confident violations (our first L1 kernel: 40/40 correct, but `np.asarray` on line 5, so zero).

**Claim: the rules belong in the verifier, not the prompt; every verdict is translated into one concrete instruction.** The model writes freely from a one-sentence task; the harness finds what is wrong; the agent sends back the previous code, *one* instruction and one line of evidence; a kernel counts only if it also passes held-out cases the model never saw.

## 2. The verifier

- **Two rule checks**: a static AST scan (covers branches the tests never reach) and a runtime guard (numpy swapped for a monitored proxy that records each violation by line).
- **Numerical correctness** against a proven float32 error bound (§5), on hostile dev cases (huge magnitudes, large means, partial tiles) and a separate holdout set.
- **Diagnosis, not just a verdict**: cross-case patterns ("wrong exactly when there is more than one column tile"), and level hypotheses checked numerically against the reference before they are said (a wrong explanation is worse than none).
- **Cost ledger**: FLOPs, instructions, HBM bytes read/written for every run (§6).
- **Final gate**: the organizers' `kernelbench.py`; its failures are translated too.

## 3. Key result: final agent vs naive

**Ours** = the final agent (version `c5e98d5a75`); naive = the same verifier and model, but the verifier's report is sent back verbatim instead of being translated into one instruction (`e22ccab872`). Holdout and stopping rule are the same. Reward per run = best attempt: 0.1 parses + 0.2 obeys rules + 0.2 runs + $0.5 \times$ fraction of cases correct (the upstream NKI agent's formula).

| Level | solved (ours / naive) | per-run reward, ours | per-run reward, naive | mean reward |
|---|---|---|---|---|
| L1 relu | **5/5** / 3/5 | 1.00 1.00 1.00 1.00 1.00 | 0.80 0.80 1.00 1.00 1.00 | **1.00** / 0.92 |
| L2 row sum | **5/5** / 0/5 | 1.00 1.00 1.00 1.00 1.00 | 0.80 0.80 0.80 0.80 0.80 | **1.00** / 0.80 |
| L3 row max | **5/5** / 1/5 | 1.00 1.00 1.00 1.00 1.00 | 1.00 0.80 0.81 0.80 0.80 | **1.00** / 0.84 |
| L4 RMSNorm | **1/5** / 0/5 | 0.81 0.81 0.81 0.61 1.00 | 0.74 0.61 0.61 0.61 0.74 | **0.81** / 0.66 |
| L5 softmax | **3/5** / 0/5 | 1.00 0.67 1.00 1.00 0.88 | 0.80 0.47 0.80 0.47 0.80 | **0.91** / 0.67 |
| L6 transpose | **5/5** / 1/5 | 1.00 1.00 1.00 1.00 1.00 | 0.80 0.80 0.80 0.80 1.00 | **1.00** / 0.84 |
| L7 matmul | **2/5** / 0/5 | 1.00 0.83 0.83 1.00 0.83 | 0.80 1.00 1.00 1.00 0.80 | **0.90** / 0.92 |
| L8 layernorm | **1/5** / 0/5 | 1.00 0.61 0.73 0.61 0.94 | 0.80 0.80 0.61 0.61 0.61 | **0.78** / 0.69 |
| L9 band attention | **4/5** / 0/5 | 1.00 1.00 1.00 0.58 1.00 | 0.80 0.80 0.52 0.80 0.80 | **0.92** / 0.74 |
| L10 conv1d | **2/5** / 4/5 | 1.00 0.35 1.00 1.00 1.00 | 1.00 0.10 1.00 1.00 1.00 | **0.87** / 0.82 |
| **Total** | **33/50** / 9/50 | | | **0.92** / 0.79 |

Naive's many 0.80s are numerically right but rule-breaking (e.g. `np.sum`): given the raw report, the model repeats the same mistake. Tokens per solved kernel: **8.5k** vs 25.2k, about 3× cheaper per success. Not every level improves: naive is better on L10.

**One failure and its recovery (L2, live log).** #1 uses `np.sum`: 28/28 correct but illegal → "accumulate column by column, `acc += x[i0:i1, j]`". #2 column loop, but indexes a 188-column partial tile with the global column: `IndexError: index 512 is out of bounds for axis 1 with size 188` (19/28 cases crash) → one instruction about the partial tile. #3 dev 28/28, holdout 21/21.

**One new diagnosis (L3).** The live bug `result[row] = acc` sat inside the column-tile loop, so each tile overwrote the last. The verifier now recomputes the reference per tile, sees the wrong outputs equal the last tile's result, and says "keep one running value per row across all tiles; write it after the loop". L3 went from stuck to 5/5.

## 4. Final kernel per level

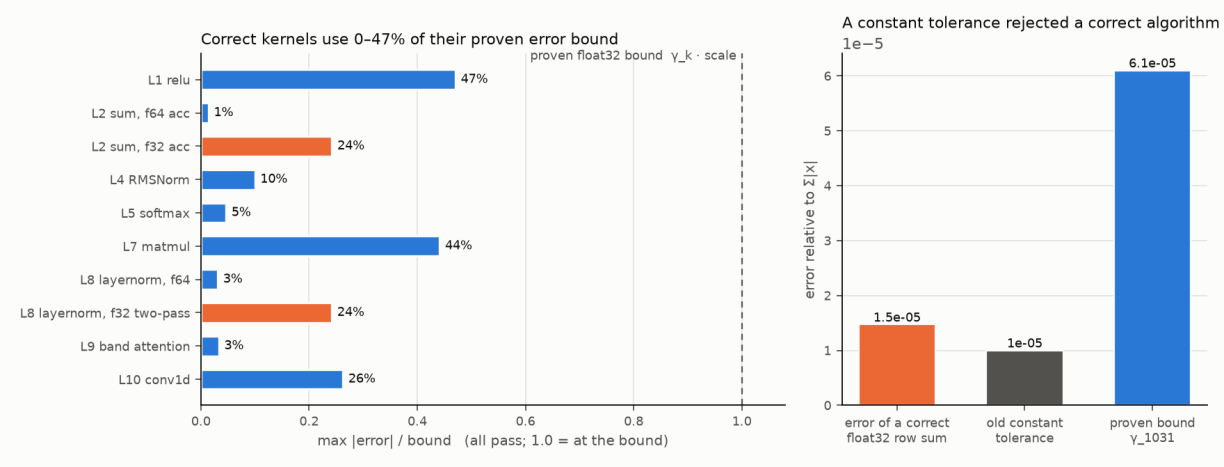
Each written by the agent and passing *both* our checker (rules, dev, holdout) and the organizers' `kernelbench.py` (`FINAL.md`, `kernels/final/`):

| | L1 | L2 | L3 | L4 | L5 | L6 | L7 | L8 | L9 | L10 |
|---|---|---|---|---|---|---|---|---|---|---|
| both checkers | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| roofline time | 1.00 | 1.00 | 1.00 | 1.50 | 2.00 | 1.00 | n/a* | 2.00 | 61.2 | 12.2 |

* written with scalar Python loops, which the cost ledger does not count; correct (organizers' 10/10, our holdout within the agent's 120 s limit) but slow, like L9 and L10.

## 5. Numerical analysis: bound the error, don't guess a tolerance

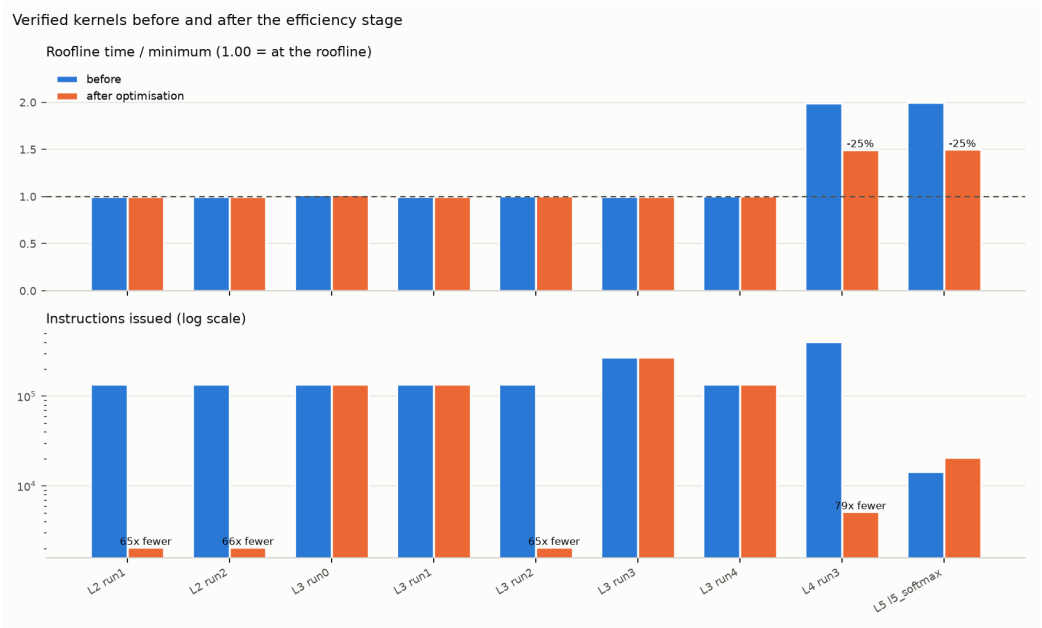
An output passes iff $|y - y_{\text{ref}}| \le \gamma_k \cdot \text{scale}$ with $\gamma_k = ku/(1-ku)$, $u = 2^{-24}$ (Higham), $k$ the longest chain of roundings (row length for a sum, $K$ for matmul, the window for attention/conv), scale the op's natural magnitude (e.g. $\sum |x|$ for a row sum). Any algorithm correct in float32 passes, whatever its order; bugs still fail by orders of magnitude.




Our first constant tolerance ($10^{-5}$) rejected a *correct* float32 row sum (error $1.5 \times 10^{-5}$; proven bound $6.1 \times 10^{-5}$). All hand kernels use 0–47% of their bound.

## 6. Efficiency: count FLOPs, bytes and instructions

The runtime guard keeps books for every run: FLOPs, instructions (numpy calls, including scalar ops), HBM bytes read and written, each against the op's minimum. Cost is the roofline time $t = \max(\text{HBM bytes}, \text{FLOPs}/222)$ as a multiple of the minimum (222 FLOP/byte is the ridge). These ops run at 0.25–0.44 FLOP/byte, so they are memory-bound: the win is fewer passes and instructions, not fewer FLOPs.




8 of 15 verified model kernels were per-element loops (∼135k instructions where column vectors need 2,068), all "correct". The optimiser gives one rule-safe instruction with a snippet and accepts a rewrite only if dev *and* holdout still pass and it is cheaper: **65× / 79× fewer instructions**, L4 roofline 1.99× → 1.50×; no kernel was ever made wrong.

## 7. Honest limits

- L4 and up are still hard: two passes + no broadcasting + numeric traps; fixing one breaks another. L4 and L8 were solved once in 5 runs, L7 and L10 twice; L7, L9 and L10 are correct but slow.
- Samples are small (5 runs per level) and the server is near-deterministic, so each run uses a different wording of the task.
- The 8k budget never bound in Stage A (largest prompt 706 tokens); it would in Stage B (NKI).

## 8. Demo instructions (about 3 minutes, no model needed)

```
git clone -b team-27-kernel-agent https://github.com/LiYou22/trainium-agent-labs.git
cd trainium-agent-labs/projects/team-27-kernel-agent && uv sync
export KERNELBENCH_DIR=$PWD/../02-kernel-agent   # organizers' kernelbench.py, in the same repo

# (a) the verifier names the cause, not just "wrong"                          -> Sec. 3, L3
uv run python -m kagent.harness 3 kernels/broken/l3_overwrite_per_tile.py --fixes
# (b) a rule violation becomes ONE instruction                                -> Sec. 3, L2
uv run python -m kagent.harness 2 kernels/broken/l2_np_sum.py --fixes
# (c) error bound and cost ledger on a correct kernel                         -> Sec. 5, 6
uv run python -m kagent.harness 4 kernels/hand/l4_rmsnorm.py
# (d) the full repair loop, replaying recorded model replies                  -> Sec. 1, 3
uv run python -m kagent.agent --levels 1 2 --scripted tests/scripts/demo.json --out /tmp/demo.jsonl
# (e) the per-run rewards of the ablation table                               -> Sec. 3
uv run python scripts/reward.py results/seat-130/ours-L1234-seat-130.jsonl \
                                 results/seat-130/naive-L1234-seat-130.jsonl
```

**(a) The verifier names the cause.** A kernel the model really wrote for L3 (row max) with `result[row] = acc` inside the column-tile loop. Point at `0 rule violation(s), 15/24 cases correct`: no rule broken, 9 cases wrong. The verifier does not stop at "wrong": `Pattern: ... each wrong output equals the result over the last column tile only` names the bug (each tile overwrites the last), which is what lets the model fix it (§3, L3).

**(b) A violation becomes one instruction.** An L2 (row sum) kernel calling `np.sum`. Point at `28/28 cases correct` *and* `np.sum [sum-like]`: numerically right, still a failure, because it calls the op it must implement. `-fixes` prints the one instruction the agent sends: `FIX line 12: Replace np.sum with an explicit python loop over the columns ...` (§3, step 1 of the L2 recovery).

**(c) Error bound and cost ledger.** Our correct hand RMSNorm. Point at `worst uses 10% of it`: the largest error is 10% of the proven float32 bound (§5); and at the `Cost` line: FLOPs, HBM reads/writes and instructions against the op's minimum (§6).

**(d) The whole loop.** The real agent loop, with recorded model replies instead of live calls, so it takes seconds and needs no server. In L2: the model's code uses `np.sum` → `FAIL rule:sum-like` → one instruction → `PASS dev; holdout 21/21` → `verified (confidence 0.95)`; 0.95 means it also passed the organizers' checker. `in=`/`out=` on each line are the tokens of that call.

**(e) The table comes from the logs.** Reads the real seat-130 run logs of both arms and prints each level's per-run reward: the same numbers as the table in §3, computed, not typed.

Order: (a) and (b) show how the verifier finds a problem and turns it into one instruction; (c) shows the numerical analysis and the cost ledger; (d) ties them into the loop; (e) shows the results are traceable. Self-tests: `uv run pytest -q` (74 passed); `README.md` has the commands for live runs.